

CAIRN Implementation Specs

Rem

August 9, 2026

Abstract

CAIRN evaluates task-agnostic metrics on an ECAN substrate: AF, STI/LTI, and Hebbian structure, sampled at CIP boundaries. Effectiveness is defined as the ratio of aggregate metric delta to composite resource cost. **Cognitive Synergy** bridges ECAN&PLN protocol evaluation (steering and feedback) on a tabulated shared map [4, 1, 5, 6]. This document gives closed forms and MeTTa-style pseudocode independent of a particular language binding.

Contents

1	Background	2
1.1	Scope	2
1.2	ECANs	2
2	Core Metrics	2
2.1	Resource Cost Functions	2
2.2	Effectiveness Calculation	3
3	Derived Metrics	4
3.1	Coherence and Retention	4
3.2	Assessment	4
3.3	Audit	5
3.4	Trajectory and Coordination	5
3.5	Benchmark	6
4	Instrumentation	7
4.1	Time Series	7
4.2	Topological Analysis	7
4.3	Utilities	8
4.4	CIP Records	8
4.5	Attention Trend Analysis	8
5	ECAN&PLN Bridge	9
5.1	Architecture	9
5.2	Protocols	10
5.3	Map sources and entry points	10
5.4	Outputs layout	11

1 Background

1.1 Scope

Structural metrics are defined only on attention state. No external task score enters those formulas. CIP evaluation compares snapshots and trajectories under a fixed agent/parameter regime.

1.2 ECANs

An ECAN can be represented as a graph consisting of the following atomic entities:

- Untyped nodes $A, B, C, \dots$ representing concepts or knowledge items
- Untyped links between nodes $d, e, f, \dots$ representing basic associations
- Typed links: HebbianLinks (H-links) and InverseHebbianLinks (IH-links)
- Weights: Short-Term Importance (STI) and Long-Term Importance (LTI)
- Assignment of STI/LTI weights to nodes
- Probability values assigned to typed links
- A forgetful operation to prune low-value atoms

STI is competitive attention mass (conserved modulo inflation and rent). LTI is retention value for keeping an atom in memory.

2 Core Metrics

2.1 Resource Cost Functions

Composite resource cost uses three normalized factors in $[0, 1]$:

- AF size ratio
- STI concentration
- AF-internal Hebbian link density

Listing 1: Ratio

```
1 (: measure-af-size-ratio (-> Number))
2 (= (measure-af-size-ratio)
3     (let* (($af-size (count (get-af-members)))
4              ($total-atoms (count (get-all-atoms-with-bins)))
5              ($ratio (/ $af-size $total-atoms)))
6         $ratio))
```

Listing 2: Concentration

```

1 (: measure-sti-concentration (-> Number))
2 (= (measure-sti-concentration)
3     (let* (($af-members (get-af-members))
4             ($af-sti (sum (map get-sti $af-members)))
5             ($total-sti (sum (map get-sti (get-all-atoms))))
6             ($concentration (/ $af-sti $total-sti)))
7         $concentration))

```

Listing 3: Density

```

1 (: measure-link-density (-> Number))
2 (= (measure-link-density)
3     (let* (($af (get-af-members))
4             ($af-size (count $af))
5             ($af-links (count-af-internal-hebbian $af))
6             ($max-links (* $af-size (- $af-size 1)))
7             ($density (if (> $max-links 0)
8                           (/ $af-links $max-links)
9                           0)))
10    $density))

```

2.2 Effectiveness Calculation

Metric delta is the mean change in STI concentration and AF link density between two CIP snapshots. Resource cost is the sum of AF size ratio, STI concentration, and AF link density at the later snapshot. Effectiveness is their ratio (0 if cost is 0).

Primary reported effectiveness is **total** (baseline CIP $\rightarrow$ current). Partial effectiveness (previous CIP $\rightarrow$ current) is recorded as a step series.

Listing 4: Attention Effectiveness

```

1 (: calculate-effectiveness (-> CIPSnapshot CIPSnapshot Number))
2 (= (calculate-effectiveness $from $to)
3     (let* (($perf-delta (aggregate-metric-delta $from $to))
4             ($resource-cost (total-resource-cost)))
5         (/ $perf-delta $resource-cost)))
6
7 (: measure-partial-effectiveness (-> CIPSnapshot CIPSnapshot Number))
8 (= (measure-partial-effectiveness $prev $curr)
9     (calculate-effectiveness $prev $curr))
10
11 (: measure-total-effectiveness (-> CIPSnapshot CIPSnapshot Number))
12 (= (measure-total-effectiveness $baseline $curr)
13     (calculate-effectiveness $baseline $curr))

```

Listing 5: Redundancy

```

1 (: test-redundancy-degradation (-> CIPSnapshot CIPSnapshot (List Atom)
  PerformanceReport))
2 (= (test-redundancy-degradation $cip-prev $cip-curr $redundant-copies)
3   (let* (($baseline-perf (aggregate-metric-delta $cip-prev $cip-curr))
4         ($baseline-cost (total-resource-cost))
5         ($_ (add-atoms-to-space $redundant-copies))
6         ($degraded-cip (transition-to-next-cip $cip-curr))
7         ($degraded-perf (aggregate-metric-delta $cip-prev $degraded-cip
8         ))
9         ($overhead (- (total-resource-cost) $baseline-cost))
10        ($degradation-rate (/ (- $baseline-perf $degraded-perf)
                                $overhead)))
    (PerformanceReport redundancy-impact $degradation-rate)))

```

3 Derived Metrics

3.1 Coherence and Retention

Listing 6: Coherence

```

1 (: measure-attention-coherence (-> Number))
2 (= (measure-attention-coherence)
3   (measure-coherence (get-af-members)))

```

Listing 7: Retention

```

1 (: measure-context-retention (-> CIPSnapshot CIPSnapshot Number))
2 (= (measure-context-retention $prev-cip $curr-cip)
3   (let* (($prev-af (get-af-snapshot $prev-cip))
4         ($curr-af (get-af-snapshot $curr-cip))
5         ($overlap (count-overlap
6         (map get-id $prev-af)
7         (map get-id $curr-af))))
8     ($union-size (+ (count $prev-af)
9                     (- (count $curr-af) $overlap))))
10    (/ $overlap $union-size)))

```

3.2 Assessment

Listing 8: Assessment

```

1 ;; Distributed importance: Pearson(mean STI series, current LTI)
2 ;; over typeSpace / bank atoms
3 (: measure-distributed-importance (-> Number))
4 (= (measure-distributed-importance)
5   (let* ($atoms (get-all-atoms))
6         ($mean-stis (map
7         (lambda ($a) (average (get-sti-series $a)))
8         $atoms))
9         ($lti-values (map get-lti $atoms)))
10    (pearson-correlation $mean-stis $lti-values)))

```

Listing 9: Assessment

```

1 ;; Selective modulation: Var(STI) / ((max-min)^2 / 12)
2 (: measure-selective-modulation (-> Number))
3 (= (measure-selective-modulation)
4     (let* (($sti-values (map get-sti (get-all-atoms)))
5             ($variance (variance $sti-values))
6             ($span (- (maximum $sti-values) (minimum $sti-values)))
7             ($null-var (/ (* $span $span) 12)))
8       (if (> $null-var 0)
9           (/ $variance $null-var)
10            0)))

```

Listing 10: Assessment

```

1 ;; Connection ratio: AF-out Hebbian with target in AF / all AF-out Hebbian
2 (: measure-connection-ratio (-> Number))
3 (= (measure-connection-ratio)
4     (let* (($from-af (get-hebbian-links-from-af))
5             ($total (count $from-af))
6             ($internal (count-if
7                          (lambda ($1) (atom-in-af (get-target $1)))
7                          $from-af)))
8       (if (> $total 0)
9           (/ $internal $total)
10            0)))

```

3.3 Audit

Listing 11: Audit

```

1 (: measure-preallocation-space (-> Number))
2 (= (measure-preallocation-space)
3     (let* (($atoms (get-all-atoms))
4             ($sti-values (map get-sti $atoms))
5             ($min-sti (minimum $sti-values))
6             ($shifted (map (lambda ($s) (- $s $min-sti)) $sti-values))
7             ($total (sum $shifted))
8             ($distribution (map (lambda ($s) (/ $s $total)) $shifted))
9             ($entropy (shannon-entropy $distribution)))
10       (/ $entropy (log2 (count $atoms)))))

```

3.4 Trajectory and Coordination

Listing 12: Trajectory

```

1 (: measure-cognitive-maintenance (-> (List CIPSnapshot) Number))
2 (= (measure-cognitive-maintenance $cip-series)
3     (let* (($pairs (consecutive-pairs $cip-series))
4             ($overlaps (map (lambda ($pair)
5                               (measure-context-retention
6                                (first $pair) (second $pair))) $pairs)))
7       (average $overlaps)))

```

Listing 13: Coordination

```

1  ;; Glocal: geometric mean of local and cross-module coherence
2  ;; Modules: weighted Louvain over all Hebbian links (weights = STV means)
3  (: measure-glocal-coordination (-> Number))
4  (= (measure-glocal-coordination)
5     (let* (($hlinks (get-hebbian-links))
6             ($modules (identify-modules $hlinks))
7             ($local-coherence (average
8                                (map measure-coherence $modules)))
9             ($inter-links (inter-module-links $hlinks $modules))
10             ($link-probs (map get-probability $inter-links))
11             ($conjunctions (map
12                             (lambda ($link)
13                               (temporal-conjunction
14                                (get-source $link)
15                                (get-target $link)))
16                             $inter-links))
17             ($global-coherence
18              (pearson-correlation $link-probs $conjunctions)))
19     (sqrt (* $local-coherence $global-coherence))))

```

3.5 Benchmark

Listing 14: Gained Efficiency

```

1  (: measure-gained-efficiency (-> CIPSnapshot CIPSnapshot
2                                CIPSnapshot CIPSnapshot Number))
3  (= (measure-gained-efficiency $base-prev $base-curr $opt-prev $opt-curr)
4     (let* (($baseline-eff (calculate-effectiveness $base-prev $base-curr))
5            ($optimized-eff (calculate-effectiveness $opt-prev $opt-curr)))
6         (/ (- $optimized-eff $baseline-eff) $baseline-eff)))

```

4 Instrumentation

4.1 Time Series

Listing 15: STI/LTI Time Series

```
1 ;; Global partial maps from atom-id to list of values
2 ;; one entry appended per CIP transition
3 (= sti-series (new-state (empty-map)))
4 (= lti-series (new-state (empty-map)))
5
6 ;; Called at each CIP boundary before AtomSpace is updated
7 (: record-cip-series (-> ()))
8 (= (record-cip-series)
9     (map (lambda ($atom)
10           (let* (($id (get-id $atom))
11                  ($curr-sti (get-sti $atom))
12                  ($curr-lti (get-lti $atom))
13                  ($prev-sti (map-get (get-state sti-series) $id))
14                  ($prev-lti (map-get (get-state lti-series) $id)))
15             (set-state sti-series
16                         (map-put (get-state sti-series) $id
17                                   (append $prev-sti $curr-sti)))
18             (set-state lti-series
19                         (map-put (get-state lti-series) $id
20                                   (append $prev-lti $curr-lti))))))
21     (get-all-atoms)))
22
23 (: get-sti-series (-> Atom (List Number)))
24 (= (get-sti-series $atom)
25     (map-get (get-state sti-series) (get-id $atom)))
26
27 (: get-lti-series (-> Atom (List Number)))
28 (= (get-lti-series $atom)
29     (map-get (get-state lti-series) (get-id $atom)))
```

4.2 Topological Analysis

Listing 16: Persistent Homology

```
1 ;; Clique-complex invariants (triangles + Betti 0/1/2 via mod-2 ranks)
2 ;; Incremental over Hebbian edge additions between CIPs
3 (: persistent-homology-invariants (-> TopologyReport))
4 (= (persistent-homology-invariants)
5     (let* (($edges (hlinks-to-edge-list))
6            ($report (topology-invariants $edges)))
7         (TopologyReport
8          (topology-triangles $report)
9          (topology-betti-0 $report)
10         (topology-betti-1 $report)
11         (topology-betti-2 $report))))
```

4.3 Utilities

Listing 17: Coherence helpers

```
1 ;; Time-averaged STI co-activation of two atoms over a CIP trajectory
2 (: temporal-conjunction (-> Atom Atom Number))
3 (= (temporal-conjunction $atom-i $atom-j)
4     (let* (($sti-i (get-sti-series $atom-i))
5             ($sti-j (get-sti-series $atom-j)))
6         (average (map * $sti-i $sti-j))))
7
8 ;; Coherence: Pearson of link probabilities vs temporal conjunctions
9 (: measure-coherence (-> (List Atom) Number))
10 (= (measure-coherence $atoms)
11     (let* (($hlinks (get-hebbian-links-within $atoms))
12            ($link-probs (map get-probability $hlinks))
13            ($conjunctions (map
14                             (lambda ($link)
15                               (temporal-conjunction
16                                (get-source $link)
17                                (get-target $link)))
18                             $hlinks)))
19     (pearson-correlation $link-probs $conjunctions)))
```

4.4 CIP Records

Listing 18: CIP Tracking

```
1 (: CIPSnapshot Type)
2 (: CIPSnapshot (->
3     Number           ; CIP index (0 = baseline)
4     Number           ; timestamp
5     (List Atom)      ; AF membership frozen at snapshot time
6     (List Metric)    ; all metrics measured at this boundary
7     CIPSnapshot))
8
9 (: initialize-baseline-cip (-> CIPSnapshot))
10 (= (initialize-baseline-cip)
11     (let* (($_ (record-cip-series))
12            ($af (get-af-members))
13            ($metrics (measure-all-metrics)))
14         (CIPSnapshot 0 (current-time) $af $metrics)))
15
16 (: transition-to-next-cip (-> CIPSnapshot CIPSnapshot))
17 (= (transition-to-next-cip $prev-cip)
18     (let* (($_ (record-cip-series))
19            ($new-index (+ (get-cip-index $prev-cip) 1))
20            ($af (get-af-members))
21            ($metrics (measure-all-metrics)))
22         (CIPSnapshot $new-index (current-time) $af $metrics)))
```

4.5 Attention Trend Analysis

Listing 19: STI Trend and Volatility

```

1  ;; STI trend for a single atom: direction of attention change at run end.
2  ;; Compares the most recent STI value to the series mean.
3  (: sti-trend (-> Atom Symbol))
4  (= (sti-trend $atom)
5     (let* (($series (get-sti-series $atom))
6            ($latest (last $series))
7            ($avg     (average $series)))
8       (if (> $latest $avg) rising
9           (if (< $latest $avg) falling
10              stable))))
11
12  ;; STI volatility: standard deviation of the atom's STI series.
13  ;; Near 0 = stable attention across CIPs; higher = fluctuating.
14  (: sti-volatility (-> Atom Number))
15  (= (sti-volatility $atom)
16     (standard-deviation (get-sti-series $atom)))

```

At end of each run trends are written to `trends.csv` (columns: atom, trend, volatility) for direct visualization, and included in `summary.json` under `atom_trends`. Three aggregate metrics derived from the per-atom data are added to `summary.json` under `af_trend_summary`:

Listing 20: Derived Attention Trend Metrics

```

1  ;; Fraction of AF atoms with stable STI trend at run end.
2  ;; Near 1 = attention fully settled; near 0 = all atoms in flux.
3  (: af-stability-ratio (-> Number))
4  (= (af-stability-ratio)
5     (/ (count-if (== trend stable) (af-trend-entries))
6        (length (af-trend-entries))))
7
8  ;; Mean STI volatility across AF atoms.
9  ;; Near 0 = low cross-CIP STI variance; higher = high turbulence.
10 (: mean-af-volatility (-> Number))
11 (= (mean-af-volatility)
12     (average (map volatility (af-trend-entries))))
13
14 ;; Net attention flow at run end: rising minus falling atoms.
15 ;; Positive = more atoms gaining salience; negative = more losing it.
16 (: attention-trajectory (-> Number))
17 (= (attention-trajectory)
18     (- (count-if (== trend rising) (af-trend-entries))
19        (count-if (== trend falling) (af-trend-entries))))
19

```

5 ECAN&PLN Bridge

This section describes the **cognitive synergy** experiment in CAIRN/bridge/.

5.1 Architecture

Attention processing. Attention data is exported to JSON (AF atoms, STI pairs, AF-incident Hebbian edges) through CIP snapshots. A separate PeTTa process tabulates a map and runs PLN. Coupling is therefore freeze-out of attention state.

Shared kernel. One measurement step is:

1. Attention snapshot $\rightarrow$ scenario generation (Hebbian edges mirrored as Inheritance premises for PeTTa PLN).
2. Pre-step builds the control focus set F (`freeze-f` or `re-dynamics`).
3. Steering runs three arms under a fixed PLN step budget hyperparameter B , recording solve, confidence, premise counts, and wall-clock `wall_ms`.

Python coordination lives in `bridge/coupled.py`, `export_snapshot.py`, `scenario_from_snapshot.py`, and `run_bridge.py`. MeTTa protocols live under `bridge/` (`steering.metta`, `feedback.metta`, `pre.metta`, `pre-freeze.metta`, `pln_api.metta`).

AF versus F . AF is the live ECAN attentional focus. F is the premise-control set used by the influenced arm. On CIP-derived maps, default `freeze-f` sets F to a top-STI subset of exported AF of size at most k (focus cap), keeping query atoms when possible. `re-dynamics` re-seeds and runs a small ECAN on the extracted graph instead of using a frozen F .

5.2 Protocols

Steering. Under fixed B , three arms share the same query Q :

- **weighted:** all premises (full table KB);
- **influenced:** premises kept by the query-conditioned filter against real focus F ($C_Q \wedge F$);
- **distracted:** same filter against a random set S with $|S| = |F|$ (seeded RNG).

Primary outputs: per-arm solved flag, confidence, n_{premises} , `wall_ms`; contrasts such as whether influenced solves when distracted does not.

Feedback. An AF- (or F -) filtered successful path yields proof-related atoms (stamp legs). Those atoms receive an STI wage and one ECAN cycle re-settles attention. Metrics include STI gain on proof objects and focus membership before/after.

Bridge-side feedback runs in the PLN process; CIP-side wage (closed loop) stimulates original CIP atom names in the live CIP process after a mid-run probe.

5.3 Map sources and entry points

Hardcoded Tabulated maps. Hand-written scenarios (`roman`, `slice`) under `bridge/scenarios/`. Driver: `run_bridge.py` (`steering` / `feedback` / suite over budgets).

CIP-derived Tabulated maps. Attention export is shared:

`export_snapshot.write_attention_snapshot` with `schedule` either at the end or midrun.

- **end-of-run:** `mve_bridge.py` exports `output/mve/bridge_snapshot.json`, then optionally runs an offline grid ($\text{modes} \times k \times B$).
- **mid-run:** `mve_pln_probe.py` injects hooks into a temporary mve; probes write `output/mve/probes/cip_i.json`.
Offline grids reuse those files without replaying the mve.
Optional closed-loop wage needs to be live.

5.4 Outputs layout

Results under `output/cognitive_synergy/`: per-cell directories named with mode, k , and B (mid-run cells include `cipi`); `summary.json` and `metrics.csv` per cell; B2 comparison table under `ablations/from_*`; B3 trajectory under `mve_pln_probe/protocol_probes.csv`.

References

- [1] Iklé, M., Pitt, J., Goertzel, B., & Sellman, G. (2010). *Economic Attention Networks: Associative Memory and Resource Allocation for General Intelligence*. Proceedings of the Third Conference on Artificial General Intelligence.
- [2] Coward, L. A. (2001). *The Recommendation Architecture: Lessons from Large-Scale Electronic Systems Applied to Cognition*. Cognitive Systems Research, 2(2), 111-156.
- [3] Sellman, G. & Goertzel, B. (2011). *Using Economic Attention Networks for Controlling Embodied Intelligent Systems*.
- [4] Goertzel, B., et al. *Using Nonlinear Dynamical Attention Allocation to Focus Probabilistic Logical Inference Upon Relevant Information*.
- [5] Finkelstein, S., Freyd, P., & Lipton, J. (1994). *Logic Programming in Tau Categories*. In: Computer Science Logic (CSL '94).
- [6] Freyd, P., & Scedrov, A. (1990). *Categories, Allegories*. North-Holland.